2.2 FRAME 팔렛 시스템 상세 분석

개요

FRAME(Framework for Runtime Aggregation of Modularized Entities)은 Substrate 런타임 개발을 위한 모듈화된 시스템입니다. FRAME은 팔렛(Pallet)이라는 모듈 단위로 구성되며, 각 팔렛은 특정 기능을 담당하는 독립적인 컴포넌트입니다. 이 문서에서는 FRAME 아키텍처와 주요 시스템 팔렛들을 상세히 분석합니다.

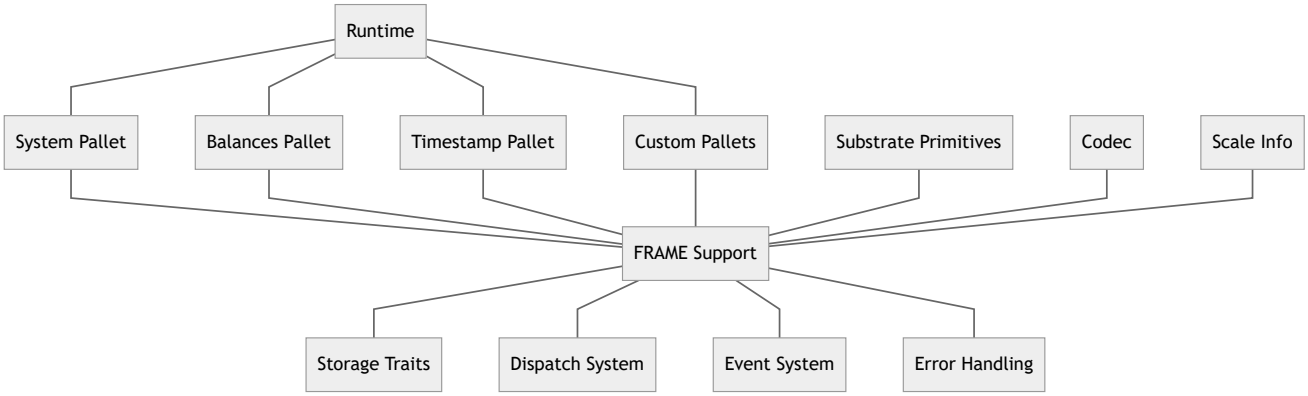
FRAME 아키텍처

팔렛 개념

팔렛은 Substrate 런타임의 기본 구성 단위로, 다음과 같은 특징을 가집니다:

- **모듈성**: 각 팔렛은 독립적인 기능을 제공
- **재사용성**: 다른 런타임에서 재사용 가능
- **구성 가능성**: 런타임에서 팔렛들을 조합하여 블록체인 구성
- **상호 운용성**: 팔렛 간 상호작용을 통한 복합 기능 구현

FRAME 구조 다이어그램



팔렛 구조 분석

기본 팔렛 구조

모든 FRAME 팔렛은 다음과 같은 기본 구조를 가집니다:

```
#[frame_support::pallet]
pub mod pallet {
use frame_support::pallet_prelude::*;
use frame_system::pallet_prelude::*;

// 1. 설정 트레이트
#[pallet::config]
pub trait Config: frame_system::Config {
type RuntimeEvent: From<Event<Self>> + IsType<<Self as frame_system::Config>::
// 추가 설정 타입들...
```

```
}

// 2. 팔렛 구조체
#[pallet::pallet]
pub struct Pallet<T>(_);

// 3. 스토리지 아이템
#[pallet::storage]
pub type SomeValue<T> = StorageValue<_, u32, ValueQuery>;

// 4. 이벤트
#[pallet::event]
#[pallet::generate_deposit(pub(super) fn deposit_event)]
pub enum Event<T: Config> {
SomethingStored { something: u32 },
}

// 5. 에러
#[pallet::error]
pub enum Error<T> {
NoneValue,
StorageOverflow,
}
```

```
// 6. 호출 가능한 함수들
#[pallet::call]
impl<T: Config> Pallet<T> {
#[pallet::weight(10_000)]
pub fn do_something(
origin: OriginFor<T>,
something: u32,
) -> DispatchResult {
let _who = ensure_signed(origin)?;
// 로직 구현...
Ok(())
}
}

// 7. 훅 (선택사항)
#[pallet::hooks]
impl<T: Config> Hooks<BlockNumberFor<T>> for Pallet<T> {
fn on_initialize(_n: BlockNumberFor<T>) -> Weight {
Weight::zero()
}
}
```

```
}
```

팔렛 구성 요소 상세 분석

1. Config 트레이트 (`#[pallet::config]`)

팔렛의 설정을 정의하는 트레이트입니다:

```
#[pallet::config]
pub trait Config: frame_system::Config {
/// 런타임 이벤트 타입
type RuntimeEvent: From<Event<Self>> + IsType<<Self as frame_system::Config>::
/// 상수 설정
#[pallet::constant]
type MaximumValue: Get<u32>;
/// 다른 팔렛에 대한 의존성
type Currency: Currency<Self::AccountId>;
/// 가중치 정보
type WeightInfo: WeightInfo;
}
```

2. 스토리지 시스템 (`#[pallet::storage]`)

FRAME은 다양한 스토리지 타입을 제공합니다:

StorageValue: 단일 값 저장

```
#[pallet::storage]
pub type SingleValue<T> = StorageValue<_, u32, ValueQuery>;
```

StorageMap: 키-값 매핑

```
#[pallet::storage]
pub type SomeMap<T: Config> = StorageMap<
_,
Blake2_128Concat,
T::AccountId,
u32,
ValueQuery
>;
```

StorageDoubleMap: 이중 키 매핑

```
#[pallet::storage]
pub type DoubleMap<T: Config> = StorageDoubleMap<
_,
Blake2_128Concat,
T::AccountId,
Blake2_128Concat,
u32,
u32,
```

```
ValueQuery
>;
```

3. 이벤트 시스템 (`#[pallet::event]`)

블록체인 상태 변경을 외부에 알리는 메커니즘:

```
#[pallet::event]
#[pallet::generate_deposit(pub(super) fn deposit_event)]
pub enum Event<T: Config> {
/// 값이 저장됨 [account, value]
ValueStored { account: T::AccountId, value: u32 },
/// 값이 제거됨 [account]
ValueRemoved { account: T::AccountId },
}
```

4. 에러 처리 (`#[pallet::error]`)

팔렛에서 발생할 수 있는 에러들을 정의:

```
#[pallet::error]
pub enum Error<T> {
/// 값이 존재하지 않음
NoneValue,
/// 저장소 오버플로우
```

```
StorageOverflow,

/// 권한 없음

NoPermission,

}
```

주요 시스템 팔렛 분석

1. System 팔렛 (`frame_system`)

목적: 런타임의 기본 기능 제공 및 다른 팔렛들의 기반 역할

주요 기능:

- 계정 관리 (nonce, 계정 정보)
- 블록 정보 저장 (블록 번호, 해시, 부모 해시)
- 이벤트 시스템 관리
- 런타임 업그레이드 처리

핵심 스토리지:

```
// 계정 정보

pub type Account<T: Config> = StorageMap<

_,

Blake2_128Concat,

T::AccountId,

AccountInfo<T::Nonce, T::AccountData>,

ValueQuery
```

```
>;


// 현재 블록 번호
pub type Number<T: Config> = StorageValue<_, BlockNumberFor<T>, ValueQuery>;


// 블록 해시
pub type BlockHash<T: Config> = StorageMap<
_,
Twox64Concat,
BlockNumberFor<T>,
T::Hash,
ValueQuery
>;
```

주요 호출 함수:

- `remark` : 온체인 메모 작성
- `set_code` : 런타임 코드 업데이트
- `set_storage` : 스토리지 직접 설정 (루트 권한 필요)

2. Balances 팔렛 (`pallet_balances`)

목적: 단일 토큰의 계정 잔액 관리

주요 기능:

- 자유 잔액 관리
- 예약 잔액 처리

- 토큰 전송
- 계정 생성 및 제거
- 총 발행량 관리

핵심 스토리지:

```
// 계정 잔액 정보
pub type Account<T: Config> = StorageMap<
_,
Blake2_128Concat,
T::AccountId,
AccountData<T::Balance>,
ValueQuery
>;

// 총 발행량
pub type TotalIssuance<T: Config> = StorageValue<_, T::Balance, ValueQuery>;
```

주요 호출 함수:

```
// 토큰 전송
#[pallet::weight(T::WeightInfo::transfer_allow_death())]
pub fn transfer_allow_death(
origin: OriginFor<T>,
dest: AccountIdLookupOf<T>,
```

```
#[pallet::compact] value: T::Balance,

) -> DispatchResult {

let  transactor = ensure_signed(origin)?;

let  dest = T::Lookup::lookup(dest)?;

<Self  as Currency<_>>::transfer(

&transactor,

&dest,

value,

ExistenceRequirement::AllowDeath,

)?;

Ok(())

}
```

트레이트 구현:

- `Currency` : 기본 통화 기능
- `ReservableCurrency` : 예약 가능한 통화
- `LockableCurrency` : 잠금 가능한 통화
- `fungible::*` : 새로운 fungible 트레이트들

3. Timestamp 팔렛 (`pallet_timestamp`)

목적: 합의 기반 시간 소스 제공

주요 기능:

- 블록 타임스탬프 설정 및 검증
- 시간 기반 로직 지원

- 합의 노드 간 시간 동기화

핵심 스토리지:

```
// 현재 타임스탬프
pub type Now<T: Config> = StorageValue<_, T::Moment, ValueQuery>;
```

Inherent 구현:

```
#[pallet::inherent]
impl<T: Config> ProvideInherent for Pallet<T> {
type Call = Call<T>;
type Error = InherentError;
const INHERENT_IDENTIFIER: InherentIdentifier = INHERENT_IDENTIFIER;

fn create_inherent(data: &InherentData) -> Option<Self::Call> {
let data: T::Moment = data
.get_data(&INHERENT_IDENTIFIER)
.expect("Timestamp inherent data not correctly encoded")
.expect("Timestamp inherent data must be provided");

Some(Call::set { now: data })
}
}
```

커스텀 팔렛 개발 가이드

1. 기본 팔렛 템플릿

```
#[frame_support::pallet]
pub mod pallet {
use frame_support::pallet_prelude::*;
use frame_system::pallet_prelude::*;

#[pallet::pallet]
pub struct Pallet<T>(_);

#[pallet::config]
pub trait Config: frame_system::Config {
type RuntimeEvent: From<Event<Self>> + IsType<<Self as frame_system::Config>::
/// 최대 저장 가능한 아이템 수
#[pallet::constant]
type MaxItems: Get<u32>;
}

// 스토리지: 사용자별 아이템 목록
#[pallet::storage]
#[pallet::getter(fn user_items)]
```

```
pub type UserItems<T: Config> = StorageMap<
_,
Blake2_128Concat,
T::AccountId,
BoundedVec<u32, T::MaxItems>,
ValueQuery
>;


#[pallet::event]
#[pallet::generate_deposit(pub(super) fn deposit_event)]
pub enum Event<T: Config> {
/// 아이템이 추가됨 [user, item_id]
ItemAdded { user: T::AccountId, item_id: u32 },
/// 아이템이 제거됨 [user, item_id]
ItemRemoved { user: T::AccountId, item_id: u32 },
}


#[pallet::error]
pub enum Error<T> {
/// 아이템 목록이 가득 찬 상태
TooManyItems,
/// 아이템을 찾을 수 없음
ItemNotFound,
```

```
}

#[pallet::call]
impl<T: Config> Pallet<T> {
/// 아이템 추가
#[pallet::weight(10_000)]
pub fn add_item(
origin: OriginFor<T>,
item_id: u32,
) -> DispatchResult {
let who = ensure_signed(origin)?;
UserItems::<T>::try_mutate(&who, |items| {
items.try_push(item_id)
.map_err(|_| Error::<T>::TooManyItems)?;
Ok(())
})?;

Self::deposit_event(Event::ItemAdded { user: who, item_id });
Ok(())
}

/// 아이템 제거
#[pallet::weight(10_000)]
```

```
pub fn remove_item(
origin: OriginFor<T>,
item_id: u32,
) -> DispatchResult {
let who = ensure_signed(origin)?;
UserItems::<T>::try_mutate(&who, |items| {
let pos = items.iter().position(|&x| x == item_id)
.ok_or(Error::<T>::ItemNotFound)?;
items.remove(pos);
Ok(())
})?;


Self::deposit_event(Event::ItemRemoved { user: who, item_id });
Ok(())
}
}
}
```

2. 팔렛 간 상호작용

팔렛들은 서로의 기능을 사용할 수 있습니다:

```
#[pallet::config]
pub trait Config: frame_system::Config {
```

```
type RuntimeEvent: From<Event<Self>> + IsType<<Self as frame_system::Config>::
/// Balances 팔렛 사용
type Currency: Currency<Self::AccountId>;
/// 다른 커스텀 팔렛 사용
type OtherPallet: SomeTraitFromOtherPallet<Self::AccountId>;
}


#[pallet::call]
impl<T: Config> Pallet<T> {
#[pallet::weight(10_000)]
pub fn paid_action(
origin: OriginFor<T>,
amount: BalanceOf<T>,
) -> DispatchResult {
let who = ensure_signed(origin)?;
// Balances 팔렛을 통한 토큰 예약
T::Currency::reserve(&who, amount)?;
// 다른 팔렛의 기능 사용
T::OtherPallet::do_something(&who)?;
Ok(())
}
}
```

베스트 프랙티스

1. 스토리지 최적화

```
// 좋은 예: BoundedVec 사용으로 스토리지 크기 제한
#[pallet::storage]
pub type BoundedItems<T: Config> = StorageMap<
_,
Blake2_128Concat,
T::AccountId,
BoundedVec<u32, T::MaxItems>,
ValueQuery
>;

// 나쁜 예: 무제한 Vec 사용
#[pallet::storage]
pub type UnboundedItems<T: Config> = StorageMap<
_,
Blake2_128Concat,
T::AccountId,
Vec<u32>,
ValueQuery
>;
```

2. 가중치 관리

```
// 벤치마킹 기반 가중치 사용
#[pallet::weight(T::WeightInfo::add_item())]
pub fn add_item(origin: OriginFor<T>, item_id: u32) -> DispatchResult {
// 구현...
}

// 복잡한 로직의 경우 동적 가중치 계산
#[pallet::weight(T::WeightInfo::complex_operation(items.len() as u32))]
pub fn complex_operation(
origin: OriginFor<T>,
items: Vec<u32>,
) -> DispatchResult {
// 구현...
}
```

3. 에러 처리

```
// 명확하고 구체적인 에러 정의
#[pallet::error]
pub enum Error<T> {
/// 아이템 목록이 최대 크기에 도달함
```

```
TooManyItems,
/// 지정된 아이템을 찾을 수 없음
ItemNotFound,
/// 권한이 없는 작업 시도
NoPermission,
/// 잔액 부족
InsufficientBalance,
}

// 적절한 에러 전파
UserItems::<T>::try_mutate(&who, |items| {
items.try_push(item_id)
.map_err(|_| Error::<T>::TooManyItems.into())
})?;
```

4. 이벤트 설계

```
// 인덱싱 가능한 이벤트 설계
#[pallet::event]
#[pallet::generate_deposit(pub(super) fn deposit_event)]
pub  enum Event<T: Config> {
/// 아이템 추가됨 [사용자, 아이템_ID, 블록_번호]
ItemAdded {
```

```
user: T::AccountId,
item_id: u32,
block_number: BlockNumberFor<T>
},
/// 대량 작업 완료 [사용자, 처리된_아이템_수]
BatchProcessed {
user: T::AccountId,
count: u32
},
}
```

런타임 통합

팔렛을 런타임에 통합하는 방법:

```
// runtime/src/lib.rs
use frame_support::{
construct_runtime,
traits::{ConstU32, ConstU64},
};

// 팔렛 설정 구현
impl my_pallet::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
```

```
type MaxItems = ConstU32<100>;

type Currency = Balances;

type WeightInfo = my_pallet::weights::SubstrateWeight<Runtime>;

}



// 런타임 구성

construct_runtime!(

pub  struct Runtime {

System: frame_system,

Timestamp: pallet_timestamp,

Balances: pallet_balances,

MyPallet: my_pallet,

}

);
```

다음 단계

이 문서에서 다룬 FRAME 팔렛 시스템을 바탕으로, 다음 문서에서는 런타임 구성과 개발에 대해 더 자세히 살펴보겠습니다:

- 2.3 런타임 구성 및 개발 가이드
- 2.4 합의 메커니즘 (BABE, GRANDPA) 구현 분석